

Ejercicios repaso examen de REACT

Ejercicios de React con el Backend

Base URL: `http://192.168.50.134:3000`

Rutas Autenticadas: Requieren `Authorization: Bearer <token>` en los headers.

◆ 1. Mostrar todos los productos con `useEffect` y `useState`

📌 **Objetivo:** Hacer una petición `GET` a `/api/products` y mostrar los productos en una lista.

📌 **Conceptos:** `useEffect`, `useState`, `fetch API`.

🏠 Tareas:

1. Crea un componente `ProductsList.jsx`.
 2. Usa `useEffect` para cargar los productos al montar el componente.
 3. Muestra los productos con `map` en tarjetas estilizadas con **Tailwind**.
-

◆ 2. Crear un contexto global para productos (`ProductsContext`)

📌 **Objetivo:** Crear un **contexto** para compartir la lista de productos en toda la app.

📌 **Conceptos:** `createContext`, `useContext`, `useState`, `useEffect`.

🏠 Tareas:

1. Crea `ProductsContext.jsx` con un `AuthProvider` que almacene los productos.
 2. Carga los productos automáticamente en el contexto con `useEffect`.
 3. Usa el **contexto** en `ProductsList.jsx` para acceder a los productos.
-

◆ 3. Implementar un formulario de login con `useAuth`

📌 **Objetivo:** Crear una página de login (`LoginPage.jsx`) que interactúe con `/api/auth/login`.

📌 **Conceptos:** `useState`, `fetch API`, `localStorage`.

🏠 Tareas:

1. Crea un formulario con **email** y **password**.

2. Al hacer submit, envía los datos a `/api/auth/login` con `fetch`.
 3. Guarda el **token** en `localStorage` y actualiza el contexto de `useAuth`.
-

◆ 4. Registrar un usuario con un formulario (`RegisterPage.jsx`)

- 📌 **Objetivo:** Crear un formulario de registro que interactúe con `/api/auth/register`.
- 📌 **Conceptos:** `useState`, `fetch` API, `useAuth`.

🏠 Tareas:

1. Crear un formulario con campos `name`, `email` y `password`.
 2. Enviar los datos a `/api/auth/register` al hacer submit.
 3. Si el registro es exitoso, iniciar sesión automáticamente.
-

◆ 5. Crear un "ProtectedRoute" para páginas privadas

- 📌 **Objetivo:** Restringir el acceso a rutas protegidas como `/dashboard`.
- 📌 **Conceptos:** `useContext`, `React Router v7`.

🏠 Tareas:

1. Crea `ProtectedRoute.jsx` que verifique si hay un usuario autenticado.
 2. Si no hay usuario, redirigir a `/login`.
 3. Usa `<ProtectedRoute>` para proteger la ruta `/dashboard`.
-

◆ 6. Crear un producto desde el frontend (`CreateProductPage.jsx`)

- 📌 **Objetivo:** Crear un producto usando `POST /api/products`.
- 📌 **Conceptos:** `useState`, `useAuth`, `fetch` API.

🏠 Tareas:

1. Crear un formulario con `name` y `price`.
 2. Enviar los datos a `/api/products` con el **token de autenticación** en los headers.
 3. Mostrar un mensaje de éxito si el producto se crea correctamente.
-

◆ 7. Editar un producto existente (`EditProductPage.jsx`)

- 📌 **Objetivo:** Actualizar un producto con `PUT /api/products/:id`.
- 📌 **Conceptos:** `useParams`, `useState`, `useEffect`.

Tareas:

1. Obtener el **id** del producto desde `useParams()`.
 2. Cargar los datos del producto con `GET /api/products/:id`.
 3. Permitir editar el nombre y precio y enviar los cambios con `PUT`.
-

◆ 8. Eliminar un producto (`DeleteProductPage.jsx`)

 **Objetivo:** Implementar una acción para eliminar un producto.

 **Conceptos:** `useParams`, `useAuth`, `fetch API`.

Tareas:

1. Obtener el **id** del producto con `useParams()`.
 2. Enviar una petición `DELETE /api/products/:id`.
 3. Redirigir a `/products` tras eliminar el producto.
-

◆ 9. Crear una página de detalles de producto (`ProductDetailPage.jsx`)

 **Objetivo:** Mostrar los detalles de un producto con `GET /api/products/:id`.

 **Conceptos:** `useParams`, `useState`, `useEffect`.

Tareas:

1. Obtener el **id** del producto con `useParams()`.
 2. Cargar la información del producto desde `/api/products/:id`.
 3. Mostrar detalles como **nombre**, **precio** y **stock**.
-

◆ 10. Crear un Dashboard con métricas de productos

 **Objetivo:** Mostrar estadísticas de los productos (`useDashboardStats`).

 **Conceptos:** `useState`, `useEffect`, `fetch API`.

Tareas:

1. Crear `useDashboardStats.js` que calcule:
 - `totalProducts`: Número total de productos.
 - `totalInventoryValue`: Suma del valor del inventario.
 - `lowStockProducts`: Productos con stock < 10 unidades.
 2. Mostrar estas métricas en tarjetas con **Tailwind**.
-

Bonus: Ideas para seguir practicando

Si completas los 10 ejercicios, prueba:

- **Paginación:** Mostrar los productos en páginas de 10 en 10.
 - **Búsqueda y filtros:** Permitir buscar productos por nombre.
 - **Subida de imágenes:** Permitir subir una imagen en `CreateProductPage.jsx`.
-